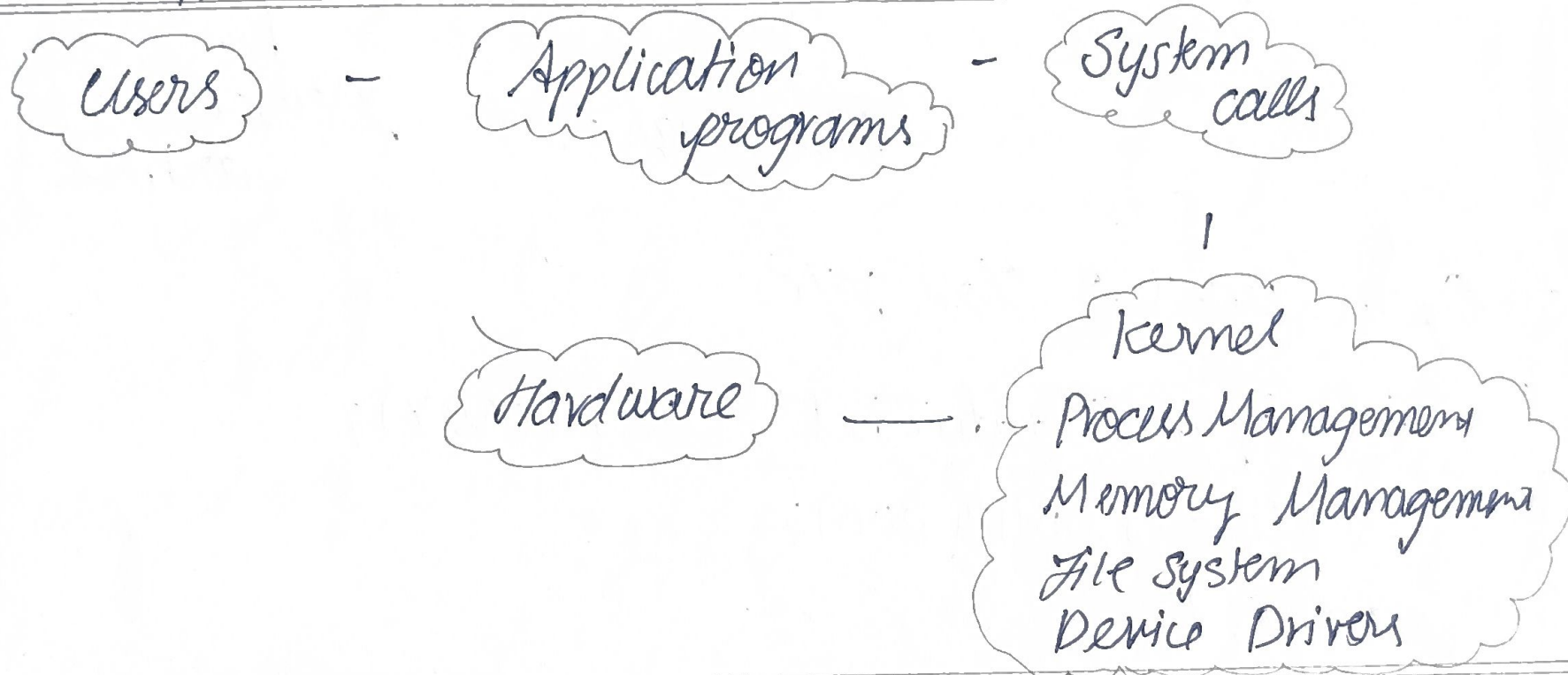


a) Types of OS suitable for this scenario

- Multi-user OS - Allows multiple users to access the system simultaneously
- Multitasking OS - Runs multiple processes at the same time.
- Time-sharing OS - Shares CPU time among users & processes efficiently.



System call Sequence

- i) fork()
- ii) exec()
- iii) wait()
- iv) exit()

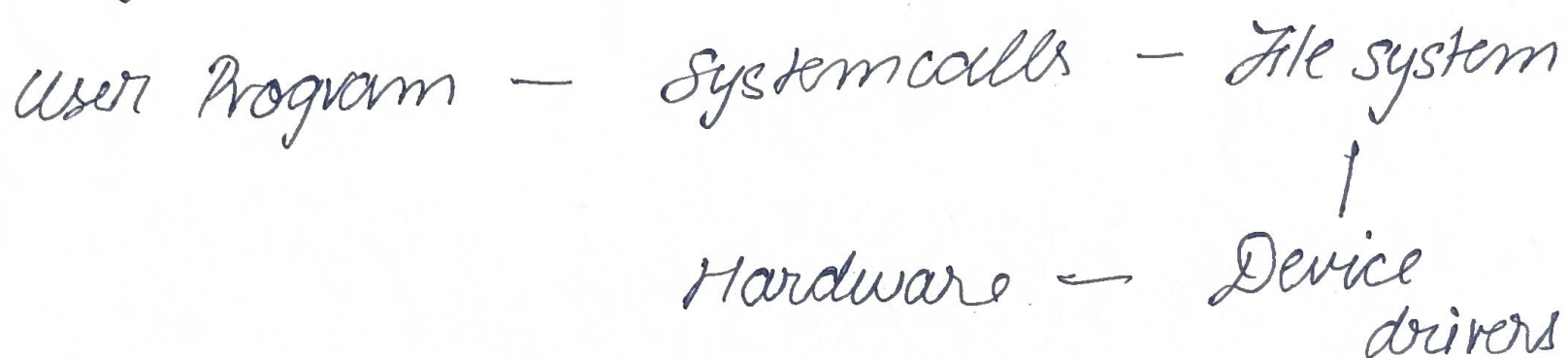
Ex:

```

pid = fork();
if (pid == 0)
    exec("program");
else
    wait()
  
```

File Management	Device Management
a) • open() • read() • write() • close() • lseek()	* ioctl() * read() * write() * open() * close()

b) Layered Structure



c) Open, Read & Close File

```

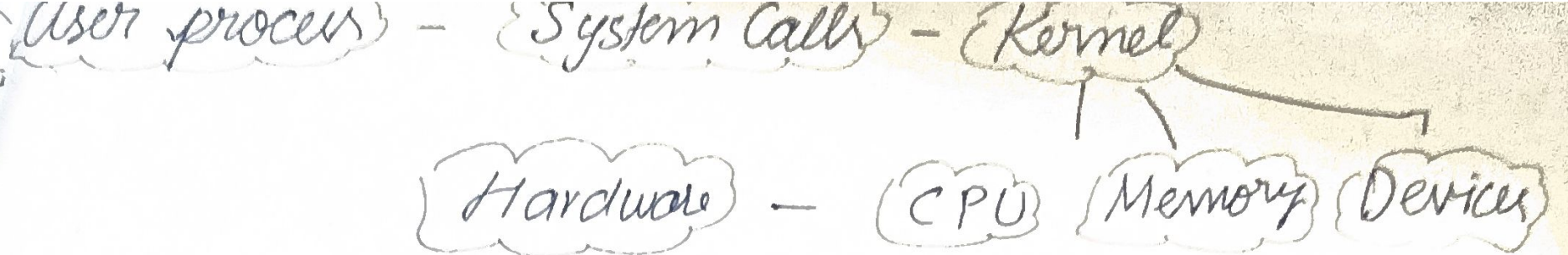
fd = open("data.txt", O_RDONLY);
read(fd, buffer, 100);
close(fd);
  
```

3.

a) Real-Time Operating System (RTOS)

Reason:

- * Fast Response
- * Predictable execution
- * Meets deadlines
- * Suitable for industrial machines



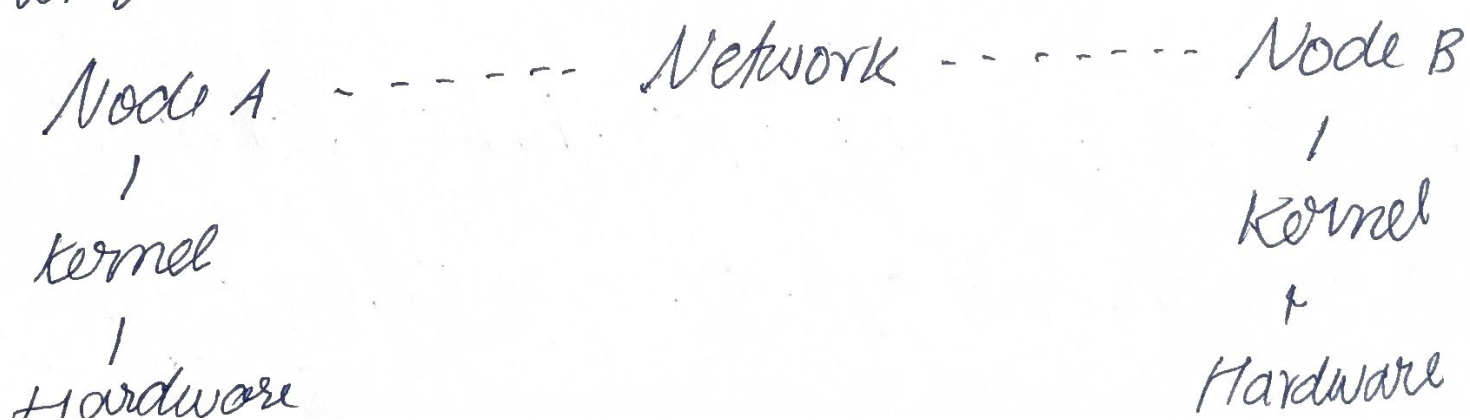
c) Synchronization System calls.

- sem_wait()
- sem_post()
- pthread_mutex_lock()
- pthread_mutex_unlock()
- wait()

These prevent race conditions & synchronizing processes

4. Distributed OS	Centralized OS
* Multiple computers * Resource sharing * Better reliability * Faster processing	* Single computer * Local resources * Single point of failure * Limited scalability

b) Architecture



1 IPC System calls

- * pipe()
- * socket()
- * send()
- * recv()
- * close()
- * write.

5. a) Major functions of an OS

- Process Management - Creates, schedules & terminate processes.
- Memory Management - Allocates and deallocates memory.
- Device Management - Controls input/output device using drivers.
- Scheduling - Shares CPU efficiently among processes.

b)

Application - System Calls

Process Management
Memory Management
File Management
Device Management
CPU scheduling

Hardware

c) Memory Allocation → mmap(), brk(), sbrk()

Device Management → open(), read(), write(), ioctl(), close()